Web Architecture & Protocols

Week 5 · Foundations module · Textbook Chapter 5

Brian C. Keegan, Ph.D.
Associate Professor, Department of Information Science
University of Colorado Boulder

September 14, 16 & 18, 2026

This week at a glance

This week we examine the layers of protocols below the web. These layers turn a URL into a page on your screen. They also turn web data collection from *magic* into *engineering*.

Companion notebook

`ch-05-protocols.ipynb`

Bring a laptop

Monday and Wednesday are both hands-on this week — bring a laptop both days.

Monday | Concepts & notebook intro

- Client–server, TCP/IP, DNS, and HTTP as one stack — and how a server decides who, or what, is asking
- Close with a live pair activity: hunt for Inspector patterns across real sites

Wednesday | Notebook lab

- Trace the stack in Python *and* the terminal: ping and traceroute from a notebook cell, a live API call, curl/wget

Friday | Textbook revisions

- Propose & peer-review pull requests improving Chapter 5

1. Monday · Concepts

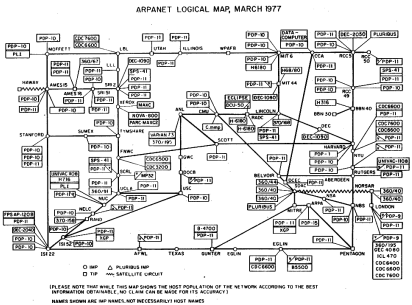
ARPANET and the bomb

Paul Baran (RAND, early 1960s) proposed **packet-switching** specifically so communications could survive a nuclear attack — no single point of failure to knock out.

ARPANET (1969, DARPA) used packet-switching, but its real job was resource-sharing between expensive and inefficient research computers

TCP/IP (Cerf & Kahn, 1970s) connected ARPANET to other, different networks, with robustness against unreliable or damaged links as a real design goal

The result is a decentralized network where packets route themselves around damage



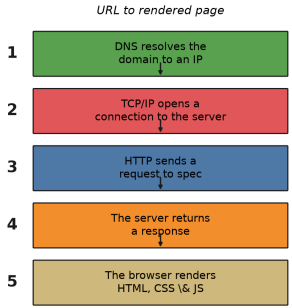
ARPANET's actual topology, March 1977.

Loading a web page, step by step

Type a URL, press Enter, and in milliseconds a precise sequence unfolds:

1. **DNS** resolves the domain name to an IP address
2. **TCP/IP** opens a connection to that server
3. **HTTP** sends a request formatted to spec
4. the server returns a **response**
5. the browser **renders** the HTML, CSS, and JavaScript

Web data science can fail at any of these levels, so debugging requires diagnoses each part.



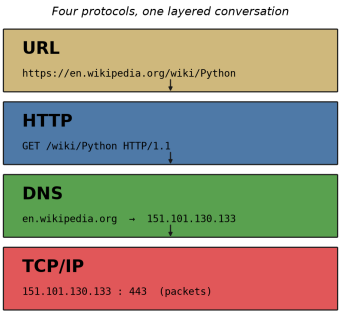
URL to rendered page, one layer at a time.

The lower stack: client–server, TCP/IP & DNS

Every scrape and API call is one **request–response** cycle: a **client** (your browser, your Python script) asks a **server** for a resource.

TCP/IP — the transport. Every machine on a public network has an IP address. Data is broken into packets, which hop across different router to router paths, and are re-assembled on arrival.

DNS — the address book. It maps a name like `en.wikipedia.org` to an IP. Names are hierarchical — `subdomain.domain.TLD` — and services follow conventions: `api.agency.gov`, `data.agency.gov`.



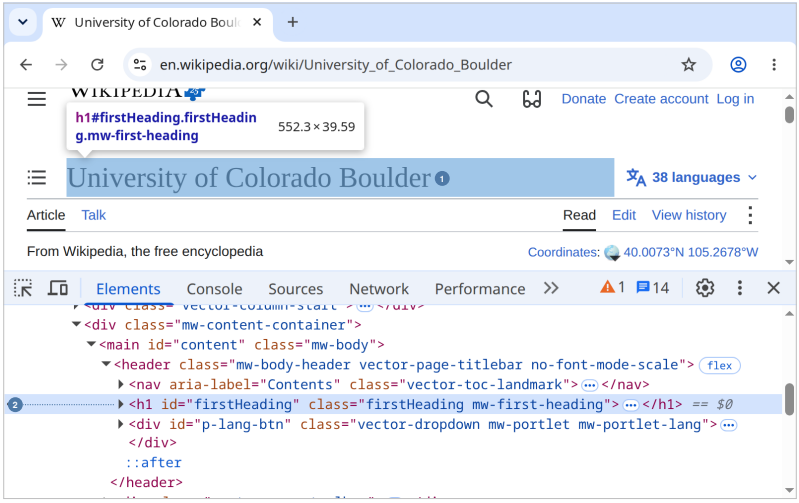
Inspecting a page: the Inspector tab

Before writing a line of code, inspect the page by hand. Open dev tools with `Ctrl+Shift+I` or `F12` (`Cmd+Opt+I` on a Mac) — or faster: **right-click the exact thing you care about** and choose **Inspect** (Chrome, Firefox) or **Inspect Element** (Safari). This jumps straight to that element instead of the top of a page that might be thousands of lines long.

- What you see is the "document object model" — the page *after* JavaScript finished making it look nicer.
- "View Source" shows the original bytes — if an element is in the Inspector but missing from `requests.get(url).text`, JavaScript put it there.
- **Hover** a node to highlight it on the page
- Right-click a node for **Copy selector**: a CSS selector to start a `soup.select()` from, though often longer than it needs to be. **Copy XPath** is for Selenium (Ch. 8) or `lxml`: `select()` can't read XPath.
- Edits here are local and temporary — safe to delete a cookie banner and see the structure beneath.

Start on a plain Wikipedia article — commercial sites bury structure under tracking scripts.

Inspecting a page: what the Inspector shows



Hover a node (2) and the browser shades its element on the page (1), labeled with its tag, `id`, classes, and size. Wikipedia, September 2026.

Inspecting requests: the Network tab

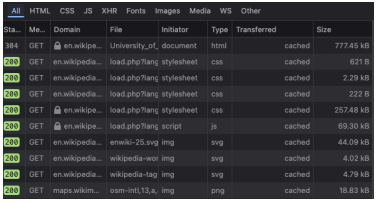
Click the **Network** tab. Reload *after* opening it.

A page load is dozens or hundreds of requests: HTML, CSS, JS, images, trackers.

Each row shows Name, Status, Type, Size, etc.

Click a request for **Headers** (incl. `User-Agent`, `Cookie`), **Response** (the raw JSON), and **Timing**.

Right-click **Copy** → **Copy as cURL** reproduces the exact request — headers, cookies, body.



One page load, dozens of requests.

Domain names aren't all `.com`

Most of this chapter's examples end in `.org`, `.gov`, or `.edu` — a U.S.-centered artifact of who built the early internet.

The U.S. has its own country-code TLD, `.us` that no one uses: `www.sos.state.co.us/`

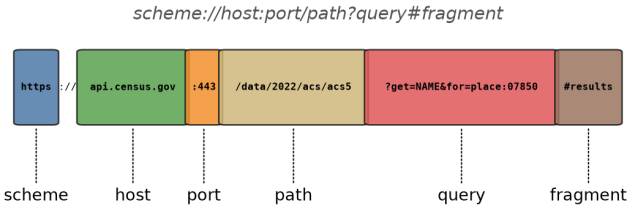
Most other countries went the opposite way: `bbc.co.uk`, `spiegel.de`, `rakuten.co.jp`

Two wrinkles to recognize

Repurposed TLDs. `.io` (British Indian Ocean Territory → "input/output"), `.ai` (Anguilla → AI startups), `.tv` (Tuvalu → streaming)

Non-Latin domains. DNS is ASCII-only but a common attack vector

The upper stack: HTTP status codes & URLs



HTTP structures the conversation: a request carries a method (GET, POST, …) and headers; a response carries a **status code**:

- **200** OK — the body has your data
- **301/302** redirect — follow `Location`
- **403** forbidden — often a missing `User-Agent`
- **404** not found **429** slow down
- **500** the server's problem, not yours

The Network tab's **Status** column already shows you these, before you open a notebook.

User-Agent spoofing

Every request so far has identified *honestly* — `WebDataScience/1.0 (you@colorado.edu)`.

Spoofing means sending a `User-Agent` that claims to be a browser or device you are not.

```
Mozilla/5.0 (Macintosh; Intel Mac OS X 10.15; rv:155.0)
Gecko/20100101 Firefox/155.0
```

Defeating a site's stated block with a spoofed user-agent has CFAA/ToS liability from Ch. 2.

A changed `User-Agent` string alone is weak and easily noticed

Identify yourself honestly; treat a block as a signal to stop



How do you do fellow browsers?

Closing activity: inspection patterns across real sites

1. Pick one plain site (a Wikipedia article, a blog) and one busy site (news, ecommerce, social feed)
2. Explore each with the Inspector: `data-*` attributes, how deep the nesting gets before you get to text, class names that look hand-written vs. auto-generated
3. Explore each with Network: how many requests are needed to load a page, what kinds of files and errors, are some requests going to other domains?
4. Find one value or attribute and edit it inside the Inspector to see it render on the page
5. Find one annoying thing — a cookie banner, a paywall overlay — and delete it live. Note what it looked like before and after

Report back with some gnarly examples



2. Wednesday · Notebook Lab

Locating yourself and the server

Your public IP is how the internet sees you; DNS is how a name becomes a routable address:

```
import requests
import dns.resolver

# Your public IP, as servers on the internet see it
print(requests.get("https://api.ipify.org?format=json").json())
# {'ip': '128.138.xxx.xxx'}

# Resolve a domain name to its IP address(es)
for rec in dns.resolver.resolve("en.wikipedia.org", "A"):
    print("IP address:", rec)
```

The terminal, inside the notebook: `!`

A line in a code cell that starts with `!` runs in your computer's terminal, and its output appears under the cell. `ping`, `traceroute`, and `curl` come with your operating system: nothing to install, no administrator rights.

```
import platform

# "Send 4 and stop": -n on Windows, -c on macOS and Linux
if platform.system() == "Windows":
    !ping -n 4 en.wikipedia.org
else:
    !ping -c 4 en.wikipedia.org
```

`ping` sends a few small packets and reports whether, and how fast, the host replies. No reply after four tries: the host is unreachable, or a firewall drops the probes.

Forgot the `-c`?

On macOS and Linux, `ping` runs until you stop it. Press the notebook's interrupt button (■).

Traceroute: reading the hops

```
import platform

# Stop after 20 hops: -h on Windows, -m on macOS and Linux
if platform.system() == "Windows":
    !tracert -h 20 en.wikipedia.org
else:
    !traceroute -m 20 en.wikipedia.org
```

Each line is one hop — an IP (or hostname) and a round-trip time. Patterns worth recognizing:

- `* * *` — an unanswered hop, not a broken path; many routers rate-limit or drop probes on purpose
- Private IPs (`10.`, `192.168.`, `172.16-31.`) in the first hop or two — your own network, normal
- A sudden RTT jump — often a long-haul or undersea link, not a problem
- Hostnames often encode geography (`iad`, `ord`) — a fast, informal location guess

It takes a minute: the tool waits for routers that don't answer. Linux without `traceroute`? Use `!tracepath`. To loop over the lines in Python: `hops = !traceroute -m 20 en.wikipedia.org`

Watching one connection: `curl -v`

Capturing **packets** ("sniffing") needs administrator rights, and on a shared network it can see other people's traffic, so we don't. `curl -v` narrates one connection from your side:

```
# -v narrates, -I asks for headers only, -A sets your User-Agent
!curl -v -I -A "WebDataScience/1.0 (you@colorado.edu)" https://en.wikipedia.org/wiki/Web_scraping
```

Read it from the top:

- `*` lines are curl's notes. By `Connected to ... port 443`, TCP's **handshake** (`SYN`, `SYN-ACK`, `ACK`) is done. The TLS handshake comes next.
- `>` lines are the request you sent: method, path, `Host`, `User-Agent`
- `<` lines are the response: the status line and the headers the Network tab showed

Want the packets themselves? **Wireshark** — on a computer you own, and only your own traffic.

Diagnosing HTTP errors, not guessing

The status code says *what* failed; the full response says *why*. Read both:

```
def diagnose_request(url, headers=None):
    try:
        r = requests.get(url, headers=headers, timeout=10)
    except requests.exceptions.ConnectionError:
        return print("Connection failed:", url)
    print("Status:", r.status_code)
    print("Content-Type:", r.headers.get("Content-Type", "n/a"))
    print("Bytes:", len(r.content))
    if r.history: # a redirect chain
        print("Redirected from:", r.history[0].url)
    return r
```

A **403** from an API usually means a missing `User-Agent` (a one-line fix); a **403** from a news site may block scraping outright. A **429** means slow down; a **500** is the server's problem — retry later.

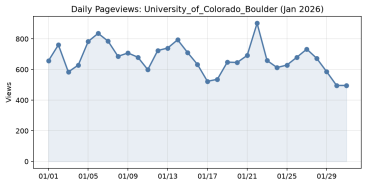
Make an API call, get a time series

The Wikimedia pageviews API rejects the default client with **403** — add a meaningful `User-Agent` and it returns **200**:

```
url = ("https://wikimedia.org/api/rest_v1/"
       "metrics/pageviews/per-article/en.wikipedia/"
       "all-access/all-agents/"
       "University_of_Colorado_Boulder/daily/"
       "20260101/20260131")
ua = {"User-Agent": "WebDataScience/1.0 (you@colorado.edu)"}
data = requests.get(url, headers=ua).json()
```

Reshape the nested JSON, then plot it:

```
df = pd.DataFrame(data["items"])
df["timestamp"] = pd.to_datetime(df["timestamp"],
                                 format="%Y%m%d00")
df.plot(x="timestamp", y="views")
```



Daily pageviews — spikes line up with real-world events.

User-Agent spoofing, mechanically

Spoofing is just the `headers` dict you already know — with a value describing a browser you are not:

```
honest = {"User-Agent": "WebDataScience/1.0 (you@colorado.edu)"}
spoofed = {"User-Agent": (
    "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 "
    "(KHTML, like Gecko) Chrome/128.0.0.0 Safari/537.36"
)}

# httpbin.org just echoes back whatever header you sent --
# a safe way to see exactly what a server receives
print(requests.get("https://httpbin.org/user-agent", headers=honest).json())
print(requests.get("https://httpbin.org/user-agent", headers=spoofed).json())
```

You already saw this header change real behavior earlier today: the pageviews API's default client got **403**; a named one got **200**. Same mechanism, different intent.

A loop over one host is your cue for a Session

A bare `requests.get()` opens a fresh connection every time. Looping over many URLs at one host? Create a `Session` — it reuses the connection, remembers headers, and retries automatically:

```
from requests.adapters import HTTPAdapter
from urllib3.util.retry import Retry

session = requests.Session()
session.headers.update({"User-Agent": "WebDataScience/1.0 (you@colorado.edu)"})

# Retry 429/5xx with backoff of 1s, 2s, 4s; honors Retry-After
retries = Retry(total=3, backoff_factor=1,
                status_forcelist=[429, 500, 502, 503, 504])
session.mount("https://", HTTPAdapter(max_retries=retries))

for article in ["Web_scraping", "Data_science"]:
    r = session.get(f"https://en.wikipedia.org/wiki/{article}")
```

Six lines of setup replace the hand-rolled backoff loop — and carry over unchanged into the scraping, Wikipedia, and government-data chapters.

The same requests, from the terminal

Every Python lookup so far has a direct terminal equivalent — no install beyond what your OS already ships:

```
dig en.wikipedia.org # or: nslookup en.wikipedia.org (any OS)
  -> same A record as dns.resolver, plus which server answered and its TTL

whois wikipedia.org
  -> a different question than DNS: who registered this, and when

curl -I https://en.wikipedia.org # headers only -- fast status check
curl -v https://en.wikipedia.org # full request/response, like the Network tab
```

In a notebook, put `!` in front of any of them. `curl` takes a User-Agent flag (`-A "..."`) the way `requests` takes a `headers` dict; DNS and WHOIS lookups don't send one.

`curl` and `wget`: alternatives to `requests`

Every tool in this section fetches; **neither parses**. The moment you need JSON, HTML, or a DataFrame, you are back to `requests`. They earn their place for a one-off check, a shell script, a server with no Python, or saving straight to disk.

`curl` is built for control (headers, methods, auth); `wget` is built for downloading (saves to disk automatically, resumes, rate-limits). Setting a User-Agent, either way:

```
curl -A "WebDataScience/1.0 (you@colorado.edu)" https://en.wikipedia.org/wiki/Web_scraping
wget --user-agent="WebDataScience/1.0 (you@colorado.edu)" https://en.wikipedia.org/wiki/Web_scraping
```

`curl` prints to your terminal by default (`-O` saves to a file); `wget` always saves to disk.

Replicating `sleep()`, and fetching many files

`wget` has rate-limiting **built in**; `curl` does not:

```
wget --wait=2 --random-wait URL1 URL2
  -> jitters 0.5x-1.5x of --wait automatically

for url in URL1 URL2; do curl -O "$url"; sleep 2; done
  -> curl needs a shell loop -- the terminal version of time.sleep()
```

A known list of files, one call, rate-limited for free:

```
wget -i articles.txt --wait=2 --random-wait \
     --user-agent="WebDataScience/1.0 (you@colorado.edu)"
```

Same task as this morning's `Session` loop over three articles — a text file of URLs instead of a Python list.

Parse — and build — URLs with the right tool

Never parse a URL with a regular expression. `urllib.parse` follows the spec:

```
from urllib.parse import urlparse, parse_qs, quote

u = urlparse("https://api.census.gov/data/2022/acs/acs5?get=NAME&for=place:07850")
u.scheme, u.netloc, u.path # 'https', 'api.census.gov', '/data/2022/acs/acs5'
parse_qs(u.query) # {'get': ['NAME'], 'for': ['place:07850']}
```

Two API styles need two builders. **Path-segment** APIs need `quote()` on each component; **query-parameter** APIs take a `params=` dict and let `requests` do the escaping:

```
quote("AC/DC", safe="") # 'AC%2FDC' -- the slash MUST be encoded
requests.get(base, params={"titles": "University of Colorado Boulder"})
```

Mixing them up — query-encoding a value the API expects in its path — is a classic source of confusing 404s.

Lab: work these, then show-and-tell

Work in pairs. Do these exercises from the chapter:

1. Open the Network tab on a Wikipedia article and on a news site. Count the requests. Give the fraction that is content and the fraction that is tracking.
2. Resolve five related domains with `dns.resolver`. Report the domains that share an IP address.
3. Plot the pageviews of three related articles. Give a possible cause for each spike.
4. Write `diagnose_url()`. Test it on a 200 response, a 404 response, a redirect, an API, and a blocked site.
5. Run the `curl -v` cell. Mark where the TCP handshake had finished, where TLS was set up, the lines you sent (`>`), and the lines you got back (`<`).
6. **5617:** examine the delivery infrastructure of one site. Record the redirect chain, the CDN headers, and the traceroute hops. Write a 500-word memo

Show-and-Tell

Bring one of these to class:

- a bug that you could not solve
- data that you collected and find interesting
- a question for a research design

3. Friday · Textbook Revisions

Improving Chapter 5 together

Today we make the book better. Each of you opens a **pull request** against `ch-05-protocols.qmd` and reviews a classmate's.

The workflow (from Week 1):

1. Branch / fork the book repo
2. Edit the chapter; commit with a clear message
3. Open a PR describing *what* and *why*
4. Review two peers' PRs: kind, specific, actionable

A strong PR

- One focused change, clearly titled; explains the problem it fixes
- Builds (`quarto render`) without errors; matches the book's voice
- Discloses any AI assistance

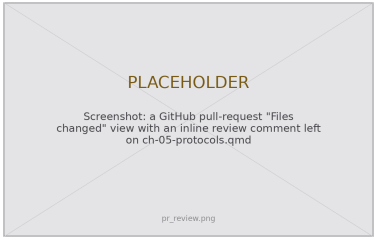
A strong review

- Runs / reads the change, not just skims; names specifics (line, claim, code)
- Separates "must fix" from "nice to have"; generous, ends with a clear verdict

A revision menu for Chapter 5

High-value targets — pick one and scope it to a single PR:

- **Test the code against live services.** `ipify`, `ip-api.com` (45 req/min), the Wikimedia pageviews API, and DNS results all drift. Run the notebook, report what broke, and fix it.
- **Add a figure.** The chapter still has zero diagrams — a layered-stack or annotated Network-tab figure would anchor the concepts.
- **Deepen "Common Issues to Debug."** An IPv6 `AAAA` lookup, and a `curl`/`wget`-specific gotcha, are still missing.
- **Strengthen an exercise.** Give `diagnose_url()` expected output, a rubric, or a starter cell so it is self-checking.
- **Add a missing citation.** ICANN, Verisign, and `ip-api.com` are all named in the chapter without a link.



A real review comment on a real PR.

Next week: **Parsing static web pages —** turning messy HTML into a clean DataFrame.

Key takeaways

1. The web is a **layered stack**: TCP/IP transports, DNS resolves names, HTTP structures the conversation, URLs identify the target.
2. **Traceroute and `curl -v`**, run from a notebook cell with `!`, make that transport layer directly observable; the `User-Agent` header — honest or spoofed — is how a server decides who, or what, is asking.
3. Debugging is **systematic diagnosis**, not guesswork: reach the server? right address? which status code? headers correct? URL well-formed?
4. A meaningful `User-Agent` fixes most **403**s; backoff or a `Retry` policy handles **429**s. A `for` loop over one host is your cue for a `Session`.
5. **Never regex a URL.** `urllib.parse` both parses and builds; know path-segment (`quote`) from query-parameter (`params=`) APIs.
6. `requests` has a terminal equivalent in `curl` and `wget` — know when opening a notebook is more overhead than the task deserves.